

# Evaluation Metrics: Inference and Claims

## Contents

Read this first — five findings that change how the paper should be framed

1. Synthetic detection correctness
2. Threshold boundary sweep
3. Prompt-size scaling (token efficiency)
4. Latency of the deterministic path
5. Evidence-grounding enforcement
6. Ablation — graph-grounded vs naive full-dataset prompting
7. Live LLM — schema validity and fallback rate
8. Live LLM — agreement with the deterministic verdict (Cohen's  $\kappa$ )
9. TAWOS, all 987 sprints (untuned rule)
10. TAWOS ranking and calibration (AUC, Brier)
11. Held-out evaluation — 270 sprints from 22 unseen projects
12. Bootstrap confidence intervals (held-out rule)
13. Negative results — three improvement attempts
14. Engineering test suites
15. Real-sprint case study — Apache Mesos sprint 74
16. Mid-sprint early warning (pace signal)
17. Paired tests at sprint end

Claims summary

Wording to fix in the existing documents

Experiments that would turn the weak claims into strong ones

For every metric: **the value, what it actually tells you (inference), what you can safely claim, and what you must not claim.** Every number was re-derived from the raw result files, not copied from the reports:

ai-service/eval/last\_run\_report.json · ablation\_result.json · live\_llm\_result\_v3.json ·  
tawos\_holdout\_bootstrap\_result.json · datasets/tawos\_score\_result.json · datasets/tawos\_holdout\_result.json

## Read this first — five findings that change how the paper should be framed

1. **Recall 1.00 on TAWOS is true by construction, not a measured result.** In the evaluation harness every task's due date is the sprint end and the rule runs one day later, so the rule predicts "delayed" whenever **at least one issue is unfinished**. The label is "delayed" when **≥30% of issues are unfinished** — which always includes at least one. So every delayed sprint is flagged automatically. This also explains why recall has a zero-width confidence interval and why the threshold sweep changed nothing.
2. **"The zero-parameter rule beats a tuned ML model" does not survive a trivial baseline.** Flagging *every* held-out sprint scores **F1 0.657**, which also beats the tuned model (0.593) and sits inside the rule's own 95% interval [0.610, 0.814]. The tuned model is actually *better* than the rule on precision, accuracy, specificity, AUC and Brier score. A paired test on the same sprints confirms it: **McNemar p = 0.78**, F1 difference +0.12 with a 95% interval of [-0.02, +0.29] (§17). The real finding is different, and still worth publishing: **the learned model did not generalise across projects.**
3. **The ablation gap (1.000 vs 0.947) is two errors and not statistically significant** (exact McNemar p = 0.5, n = 36). Claim *no loss of accuracy, not more accurate*.
4. **The architecture claims are the strong ones.** Bounded prompt size, mechanically enforced citations, and the LLM not overriding computed verdicts are all well supported. Lead with these.
5. **There is now a genuine real-data prediction result.** Halfway through a sprint, knowing only what had happened by then, the new pace signal ranks the sprints that will end delayed with **AUC 0.79** on 22 unseen projects (**0.87** at three-quarters), and beats flagging every sprint significantly (§16). Use this as the real-world headline; keep the end-of-sprint results as a sanity check.

## 1. Synthetic detection correctness

| Metric                             | Value                                              |
|------------------------------------|----------------------------------------------------|
| Scenarios                          | 180 (90 positive / 90 negative, 30 per risk type)  |
| TP / FP / FN / TN                  | 90 / 0 / 0 / 90                                    |
| Precision / Recall / F1 / Accuracy | 1.00 / 1.00 / 1.00 / 1.00                          |
| Stability                          | identical across 4 more seeds (1,200 extra trials) |

**Inference.** The anomalies are injected to match each detector's own definition, so a correct implementation *must* score 100%. This is the equivalent of "all unit tests pass" for the six detectors. It says nothing about how often these patterns predict real trouble.

**Claim.** "All six graph detectors are implemented correctly: on 180 scenarios with known ground truth they produce no false positives or false negatives, and the result is identical across five random seeds."

**Do not claim.** Accuracy, generalisation, or real-world detection performance.

## 2. Threshold boundary sweep

| Signal        | Documented threshold | Observed flip                 |
|---------------|----------------------|-------------------------------|
| overdue_ratio | 0.10                 | exactly 0.10                  |
| silent_ratio  | 0.30                 | exactly at 0.30 (0.20 → 0.30) |
| workload_skew | 1.50                 | between 1.429 and 1.522       |

**Inference.** This is the falsifiable version of §1: each detector switches exactly where the paper says it does. The workload result is a bracket only because the sweep steps don't land on 1.50 exactly, not because the detector is off.

**Claim.** "Detector thresholds are auditable: each signal changes verdict at its documented value."

**Do not claim.** That these thresholds are *optimal*. They were chosen, not tuned.

## 3. Prompt-size scaling (token efficiency)

| Tasks | Graph-grounded (witness nodes) | Naive full dataset | Ratio  |
|-------|--------------------------------|--------------------|--------|
| 13    | 266 (6)                        | 1,050              | 3.9×   |
| 53    | 386 (9)                        | 4,695              | 12.2×  |
| 203   | 389 (9)                        | 18,901             | 48.6×  |
| 1,003 | 389 (9)                        | 94,724             | 243.5× |
| 2,003 | 390 (9)                        | 189,834            | 486.8× |

**Inference.**

- The naive payload grows linearly, about **95 tokens per task**. The graph payload plateaus from 53 tasks onward, because the witness subgraph stays at 9 nodes.
- Project size grows 154×; the graph payload grows 1.47×, almost all of it between 13 and 53 tasks.
- The counts are **estimates (characters ÷ 4)**, applied the same way to both sides, so the *ratio* is reliable even though the absolute counts are approximate.
- The counts cover the **project-data part of the prompt only**. Fixed instructions add roughly 700–850 tokens (from the ablation's full prompts). Adding that overhead to both sides gives about **170× at 2,003 tasks** for the whole prompt. This is an estimate, not a measurement.
- The sweep holds the number of anomalies fixed. The bound as anomalies grow comes from the design cap ( $\leq 8$  neighbours per finding,  $\leq 60$  nodes), which was not swept.

**Claim.** "The project data sent to the language model stays essentially constant as the project grows: 1.47× more tokens for a 154× larger project. At 2,003 tasks it is 487× smaller than sending the full dataset (character-based estimate; project-data payload)."

**Do not claim.**

- A 487× reduction in the *total* prompt or in cost.
- Tokenizer-exact counts.
- An empirically measured  $O(\text{anomalies})$  scaling. That property follows from the design cap; it was not measured by varying the number of anomalies.

## 4. Latency of the deterministic path

| p50      | p95      | mean     | max      | Setup                                                        |
|----------|----------|----------|----------|--------------------------------------------------------------|
| 10.96 ms | 13.42 ms | 11.87 ms | 50.58 ms | 203 tasks, n = 50, graph build + metrics + witness selection |

**Inference.**

- The distribution is tight (p95 is  $1.2 \times p50$ ). The 50 ms maximum is a single outlier, most likely warm-up.
- p50 varied between 4.8 and 11 ms across runs, so it depends on machine load.
- Only one project size was timed.
- This times the **graph computation**, not a full analysis request. In the running prototype on the development machine, a complete analysis request (database snapshot, HTTP calls, all agents) took **9–12 s**. That is an informal observation with verbose logging enabled, not a benchmark.

**Claim.** "Computing all six risk scores and the witness subgraph takes about 11 ms for a 203-task project, fast enough to run synchronously inside a request."

**Do not claim.**

- That a full analysis takes 11 ms.
- Any scaling of latency with project size (not measured).

## 5. Evidence-grounding enforcement

| Metric                        | Value            |
|-------------------------------|------------------|
| Fabricated references removed | 200 / 200 (100%) |

**Inference.** Grounding is a set-membership check: a citation survives only if its node id is in the witness subgraph. 100% is what a correct implementation must produce. The test shows the check is wired into the pipeline and cannot be bypassed. The guarantee is narrow: a **real but irrelevant** node would still pass.

**Claim.** "No fabricated evidence reference can reach the user: every citation is mechanically checked against the graph nodes the agent was shown, and all 200 injected fabrications were removed."

**Do not claim.** That explanations are correct, relevant or useful. That needs a human evaluation, which has not been done.

## 6. Ablation — graph-grounded vs naive full-dataset prompting

| Architecture             | TP / FP / FN / TN | Precision | Recall | F1    | Accuracy |
|--------------------------|-------------------|-----------|--------|-------|----------|
| Graph-grounded           | 18 / 0 / 0 / 18   | 1.000     | 1.000  | 1.000 | 1.000    |
| Naive LLM (gpt-oss-120b) | 18 / 2 / 0 / 16   | 0.900     | 1.000  | 0.947 | 0.944    |

Per risk type: identical on 5 of 6; both of the naive model's false positives are **coordination** (precision 0.60). Mean prompt size: 1,109 vs 1,804 tokens.

**Inference.**

- The whole difference is **2 false positives**. On paired outcomes that gives exact McNemar  $p = 0.5$ : no statistically significant difference.
- The naive LLM is good on small projects. It reports 2.75 risk types per scenario against the graph's 2.42, and only 12.1% of its claims are uncorroborated.
- Ground truth comes from the same definitions the graph detectors implement, so the graph side is expected to score 100%. The test measures whether an LLM reading raw data *agrees with* those definitions.
- The projects are deliberately tiny (10–20 tasks), which favours the naive approach. The token gap here is only  $1.6\times$ , not the  $487\times$  from §3.

**Claim.** "Replacing full-dataset prompting with graph-computed findings costs no detection accuracy on 36 matched scenarios (F1 1.000 vs 0.947; the two false positives, both on coordination, are not a significant difference). The graph approach is also deterministic and cheaper."

**Do not claim.**

- That graph-grounding is *more accurate* than an LLM.
- Anything about naive-LLM accuracy on large projects (untested).

## 7. Live LLM — schema validity and fallback rate

| Metric                                      | Value                                                                                               |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Valid structured responses, all attempts    | 41.1% (37 / 90)                                                                                     |
| Fallback used                               | 58.9% (progress 21/30, planning 16/30, workload 16/30)                                              |
| Failures                                    | 53: 24 diagnosed (22 were HTTP 429 rate limits), 29 undiagnosed (from an earlier, checkpointed run) |
| Validity among calls that reached the model | between 54% and 95%, depending on the undiagnosed 29                                                |

**Inference.**

- On a free-tier key, most failures were rate limits, not bad model output.
- The report's "92%" assumes the 29 undiagnosed failures were also rate limits. That is likely, since 22 of the 24 diagnosed ones were, but it was not measured.
- Every analysis still completed: when the model call failed, the deterministic fallback produced the result.

**Claim.**

- "Under a rate-limited free API key, 41% of language-model calls returned valid structured output end-to-end. Most failures were HTTP 429 rate limits (22 of 24 diagnosed)."
- "The deterministic fallback produced a complete result for every analysis."

**Do not claim.** "The model produces valid output 92% of the time" as a measured figure. Say "estimated 54–95%, most likely near the top" or re-run with full diagnostics.

## 8. Live LLM — agreement with the deterministic verdict (Cohen's $\kappa$ )

| Overall                                          | n  | By risk type                                                                                                              |
|--------------------------------------------------|----|---------------------------------------------------------------------------------------------------------------------------|
| $\kappa = 0.953$ ("almost perfect", Landis–Koch) | 37 | dependency 1.00 (n=14) · knowledge 0.86 (n=12) · workload 1.00 (n=8) · coordination, delay, silent member 1.00 (n=1 each) |

**Inference.**

- When the model answers, its risk level matches the deterministic verdict almost every time.
- The model is *shown* the computed finding. So  $\kappa$  measures whether the narration layer **respects** the computed verdict, not whether an LLM independently reaches it.
- The sample is small and uneven: three risk types have one comparison each.
- Only the 37 valid responses are compared. If invalid responses were more likely to disagree,  $\kappa$  would be optimistic.

**Claim.** "The language model narrates rather than re-decides: on 37 valid responses its risk level agreed with the deterministic verdict with  $\kappa = 0.95$ ."

**Do not claim.**

- Per-risk-type agreement for coordination, delay or silent members (n = 1).
- That an LLM can detect these risks on its own.
- Generalisation to other models or providers.

## 9. TAWOS, all 987 sprints (untuned rule)

| Model                       | TP / FP / FN / TN     | Precision | Recall | F1    | Accuracy |
|-----------------------------|-----------------------|-----------|--------|-------|----------|
| Graph rule                  | 580 / 341 / 0 / 66    | 0.630     | 1.000  | 0.773 | 0.655    |
| Story-point baseline        | 224 / 187 / 356 / 220 | 0.545     | 0.386  | 0.452 | 0.450    |
| Flag every sprint (derived) | 580 / 407 / 0 / 0     | 0.588     | 1.000  | 0.740 | 0.588    |

Prevalence 58.8% · rule flags **93.3%** of sprints · specificity **16.2%** · per-project F1 ranges from **0.27 to 1.00** across 31 projects.

### Inference.

- Recall 1.00 is structural (see "Read this first").
- What the data does show: **84% of on-time sprints still end with at least one unfinished issue** (341 of 407). "Something is unfinished" therefore separates delayed from on-time sprints only weakly.
- The story-point baseline scores below even "flag every sprint", so beating it proves little.
- Performance varies widely between projects.

### Claim.

- "On 987 real Jira sprints reconstructed as of sprint end (31 projects, no future leakage), the delay signal beats a flag-every-sprint baseline modestly (F1 0.773 vs 0.740; accuracy 0.655 vs 0.588)."
- "Its sensitivity is guaranteed by how the signal and the label are defined."

### Do not claim.

- "Catches every delayed sprint" as evidence of detection skill.
- That beating the story-point baseline is meaningful.

## 10. TAWOS ranking and calibration (AUC, Brier)

| Metric                   | Graph rule (all 987)    | Reference                                                     |
|--------------------------|-------------------------|---------------------------------------------------------------|
| AUC-ROC                  | 0.581                   | random = 0.5                                                  |
| Brier score              | 0.346                   | a constant base-rate forecast = 0.242; a constant 0.5 = 0.250 |
| Held-out composite model | AUC 0.658 · Brier 0.261 | held-out base-rate forecast = 0.250                           |

### Inference.

- At sprint end the delay score is effectively on/off, so it carries almost no ranking information: AUC 0.58 is barely above chance.
- Read as a probability, the severity score is **worse calibrated than always predicting the base rate**. The tuned composite is also slightly worse than that constant forecast on held-out data.

**Claim.** "The sprint-level delay score is a binary warning, not a calibrated probability (AUC 0.58; Brier 0.35, worse than a constant forecast). Its severity should be read as a rank-free flag."

**Do not claim.** That confidence values are calibrated probabilities, or that the score ranks risk well.

## 11. Held-out evaluation — 270 sprints from 22 unseen projects

| Model                       | TP / FP / FN / TN  | Precision | Recall | F1    | Accuracy | Specificity | Flags | AUC   | Brier |
|-----------------------------|--------------------|-----------|--------|-------|----------|-------------|-------|-------|-------|
| Graph rule (0 parameters)   | 132 / 107 / 0 / 31 | 0.552     | 1.000  | 0.712 | 0.604    | 0.225       | 88.5% | —     | —     |
| Tuned composite (logistic)  | 75 / 46 / 57 / 92  | 0.620     | 0.568  | 0.593 | 0.619    | 0.667       | 44.8% | 0.658 | 0.261 |
| Flag every sprint (derived) | 132 / 138 / 0 / 0  | 0.489     | 1.000  | 0.657 | 0.489    | 0.000       | 100%  | —     | —     |

Prevalence 48.9%.

#### Inference.

- The rule "wins" only on F1 and recall. F1 favours recall-heavy predictions when about half the sprints are positive: even flagging everything beats the composite.
- The composite is more precise, more accurate, and three times better at clearing on-time sprints. But it misses **57 of 132 delayed sprints (43%)**.
- The two models sit at very different operating points. Neither dominates, and the composite's 0.5 threshold is clearly badly placed for F1.
- The rule beats "flag every sprint" by F1 +0.055 and accuracy +0.115, by correctly clearing 31 on-time sprints.

#### Claim.

- "On 22 projects never used for design or tuning, the zero-parameter rule is a high-sensitivity early warning (no missed delays, by construction) that correctly clears 22% of on-time sprints."
- "A tuned four-feature model trades recall for precision (0.620 vs 0.552) and misses 43% of delayed sprints."
- "Neither dominates; the rule is preferable when a missed delay costs more than a false alarm, and it is fully explainable."

**Do not claim.** "A zero-parameter rule beats a tuned ML model". That is only true on F1, and a flag-everything baseline also beats that model on F1.

## 12. Bootstrap confidence intervals (held-out rule)

| Metric    | Point | 95% CI (cluster bootstrap by project, 2,000 resamples) | Bootstrap SD |
|-----------|-------|--------------------------------------------------------|--------------|
| F1        | 0.712 | [0.610, 0.814]                                         | 0.051        |
| Precision | 0.552 | [0.438, 0.686]                                         | 0.062        |
| Recall    | 1.000 | [1.000, 1.000]                                         | 0            |
| Accuracy  | 0.604 | [0.517, 0.719]                                         | 0.051        |

#### Inference.

- The F1 estimate moves by about  $\pm 0.10$  depending on which projects are sampled. Resampling by project is the right choice, because sprints within a project are correlated.
- The zero-width recall interval follows from the label definition, not from robustness.
- Comparing this interval's lower bound (0.610) with the composite's *point* estimate (0.593) is not a valid test. The composite has its own uncertainty, and both models predict the same sprints, so a **paired** test is needed.
- The flag-everything baseline (0.657) lies **inside** the interval, so there is no evidence the rule beats it on F1.

**Claim.** "The held-out F1 of 0.712 has a 95% project-level bootstrap interval of [0.610, 0.814]."

#### Do not claim.

- That the interval "confirms the rule beats the tuned model".
- That recall is robust because its interval is tight.

## 13. Negative results — three improvement attempts

| Attempt                                              | Result                                                | Inference                                                                                                                                                                        |
|------------------------------------------------------|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Threshold sweep 0.05–0.50                            | Identical output at every value                       | Expected: the overdue ratio is exactly 0 or 1 at sprint end, so no threshold between them changes anything. This confirms the signal is binary.                                  |
| Individualised due dates (2.96 days per story point) | AUC 0.581 → 0.531                                     | Story points are too noisy a stand-in for duration to place per-task deadlines.                                                                                                  |
| Composite logistic model, 3-fold group CV by project | CV F1 0.816, AUC 0.887 → held-out F1 0.593, AUC 0.658 | A large drop from cross-validation to unseen projects means the model learned project-specific structure. Its top feature (idle-member ratio) is mechanically tied to the label. |

**Claim.** This is the most defensible real-data finding: "A learned composite that looked strong under project-grouped cross-validation (F1 0.816, AUC 0.887) did not transfer to unseen projects (F1 0.593, AUC 0.658). Per-project calibration of delay models is a real obstacle, and complexity alone did not help."

**Do not claim.** "Complexity is actively harmful" as a general law. This is one dataset, one feature set and one model family.

## 14. Engineering test suites

| Suite      | In the reports | Current |
|------------|----------------|---------|
| Backend    | 29             | 52      |
| AI service | 43             | 59      |
| Frontend   | 27             | 41      |

**Inference.** These are engineering evidence (the system works and regressions are caught), not research evidence. The report numbers are out of date.

**Claim.** "The implementation is covered by 152 automated tests across the three services." Keep this in a methods or artifact section, not the results.

## 15. Real-sprint case study — Apache Mesos sprint 74

- **Inputs:** 30 real issues, 13 assignees, 2 blocking links, 71 comments.
- **Verdict:** risk critical; 20 of 30 issues unfinished at sprint end; workload skew 2.34 (one contributor with 9 open points against a team mean of 3.8).
- **Apply and re-run:** the three suggested reassignments were applied and the analysis re-run. Workload went from **high to medium**, as the system predicted.

**Inference.** This is a single qualitative walkthrough ( $n = 1$ ). It shows the full loop works on real data (detect, explain with cited evidence, act, re-measure). It is not evidence that the suggestions improve real outcomes; the "improvement" is measured by the same metric that produced the suggestion.

**Claim.** "An illustrative case study on a real Apache Mesos sprint demonstrates the complete loop, including a suggested rebalancing whose predicted effect was confirmed on re-analysis."

**Do not claim.** That applying recommendations reduces real delay or improves team outcomes (no longitudinal or user study).

## 16. Mid-sprint early warning (pace signal)

**What was added.** The graph metrics now project each milestone's burn-down: projected unfinished work at the deadline =  $1 - (\text{share of work done} \div \text{share of the schedule elapsed})$ , weighting tasks by story points with a minimum of 1. The projection runs between 25% of the schedule and the deadline, and is graded with the existing 10% / 25% / 40% cut-offs. The evaluation runs at **50% and 75%** of each sprint using only what existed by then: issues created by the checkpoint count as in scope, and issues resolved by the checkpoint count as done. The label is unchanged. Everything was decided before any result was seen and nothing is tuned (eval/datasets/tawos\_midsprint\_eval.py).

Held-out: 270 sprints, 22 unseen projects, prevalence 48.9%.

| Checkpoint | Warning level                           | Precision | Recall | F1    | Accuracy | Specificity | Flags |
|------------|-----------------------------------------|-----------|--------|-------|----------|-------------|-------|
| 50%        | any ( $\geq 10\%$ projected unfinished) | 0.581     | 0.901  | 0.706 | 0.633    | 0.377       | 75.9% |
| 50%        | high or critical ( $\geq 25\%$ )        | 0.626     | 0.849  | 0.720 | 0.678    | 0.514       | 66.3% |
| 75%        | any                                     | 0.623     | 0.962  | 0.756 | 0.696    | 0.442       | 75.6% |
| 75%        | high or critical                        | 0.719     | 0.909  | 0.803 | 0.781    | 0.659       | 61.9% |
| —          | flag every sprint                       | 0.489     | 1.000  | 0.657 | 0.489    | 0.000       | 100%  |

| Checkpoint | AUC, projected unfinished | AUC, share of issues still open | Paired vs flag every sprint                              |
|------------|---------------------------|---------------------------------|----------------------------------------------------------|
| 50%        | 0.792                     | 0.802                           | McNemar $p < 0.001$ · F1 +0.050, 95% CI [+0.006, +0.094] |
| 75%        | 0.874                     | 0.881                           | McNemar $p < 0.001$ · F1 +0.099, 95% CI [+0.049, +0.148] |

The tuning split (717 sprints, 9 projects) shows the same pattern: AUC 0.819 and 0.870, and a high-or-critical F1 of 0.821 and 0.848.

### Inference.

- This is the first real predictive result. Recall is no longer guaranteed (0.85–0.96 depending on checkpoint and warning level). The score ranks sprints well: AUC 0.79 halfway and 0.87 at three-quarters, against 0.58 for the end-of-sprint score.
- It significantly beats flagging every sprint on unseen projects at both checkpoints, and it improves as the sprint progresses, as a real forecast should.
- The projection ranks **no better than the plain share of issues still open** (0.79 vs 0.80, 0.87 vs 0.88). At the same point in every sprint the two carry the same information. The projection's value is that it adjusts for elapsed time, so one threshold works at any moment of a live sprint.
- The high-or-critical level is the better operating point at both checkpoints. The tuning split shows the same ordering, so preferring it does not depend on the held-out data.
- Scope comes from issue creation dates. An issue created before the checkpoint but added to the sprint later counts as in scope, so the checkpoint view is not perfectly point-in-time.

### Claim.

- "Halfway through a sprint, a zero-parameter burn-down projection on the project graph identifies the sprints that will end delayed on 22 unseen projects: AUC 0.79, and recall 0.85 at precision 0.63 for the high-severity warning."
- "It is significantly better than flagging every sprint (paired McNemar  $p < 0.001$ ). At three-quarters of the sprint, AUC rises to 0.87 (F1 0.80, accuracy 0.78)."

### Do not claim.

- That the graph adds predictive power beyond the share of work still open (AUC 0.79 vs 0.80).
- Exact point-in-time scope: issues can join a sprint after they were created.

## 17. Paired tests at sprint end

The same 270 held-out sprints and the same predictions as §11, compared pairwise:

| Comparison                | Only A right / only B right | McNemar p | F1 difference, 95% CI (cluster bootstrap) |
|---------------------------|-----------------------------|-----------|-------------------------------------------|
| Rule vs tuned composite   | 57 / 61                     | 0.78      | +0.119 [−0.025, +0.291]                   |
| Rule vs flag every sprint | 31 / 0                      | < 0.001   | +0.055 [+0.030, +0.080]                   |

### Inference.

- The rule and the tuned model cannot be told apart. Each gets a similar number of sprints right, but on different sprints, and the F1 interval includes zero.
- The rule does beat flagging every sprint, narrowly but reliably. The 31 on-time sprints it clears are its entire advantage.

**Claim.** "At sprint end, the zero-parameter rule and the tuned composite are statistically indistinguishable on unseen projects (McNemar  $p = 0.78$ ). The rule significantly outperforms flagging every sprint (F1 +0.055, 95% CI [+0.030, +0.080])."

**Do not claim.** That the rule beats the tuned model.

## Claims summary

| #   | Claim                                                                                                      | Status                                                                          | Evidence                                           |
|-----|------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|----------------------------------------------------|
| N1  | Risks computed as graph properties; LLM only explains                                                      | Supported                                                                       | §1, §2, §6 (no accuracy loss), §8 ( $\kappa$ 0.95) |
| N2  | Project-data prompt size stays flat as projects grow                                                       | Supported (payload only, character estimate)                                    | §3                                                 |
| N3  | Fabricated evidence cannot reach the user                                                                  | Supported (mechanical guarantee, narrow scope)                                  | §5                                                 |
| N4  | Zero-parameter rule beats a tuned ML model on unseen data                                                  | Not supported (paired $p = 0.78$ )                                              | §9, §11, §12, §17                                  |
| N4' | Learned composite failed to generalise across projects; explainable rule is a strong recall-first baseline | Supported                                                                       | §11, §13, §17                                      |
| N5  | A graph burn-down projection warns of delayed sprints halfway through                                      | Supported (AUC 0.79 at 50%, 0.87 at 75%; beats flag-every-sprint, $p < 0.001$ ) | §16                                                |
| —   | Graph-grounding is more accurate than an LLM                                                               | Not supported ( $p = 0.5$ )                                                     | §6                                                 |
| —   | Every delayed sprint is caught                                                                             | True but structural — do not present as skill                                   | §9, §12                                            |
| —   | LLM gives valid output ~92% of the time                                                                    | Estimate only (54–95%)                                                          | §7                                                 |
| —   | Analysis runs in ~11 ms                                                                                    | Graph computation only                                                          | §4                                                 |

## Wording to fix in the existing documents

| Where                                          | Current                                             | Suggested                                                                                           |
|------------------------------------------------|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| README "Key Results", IEEE report §4 Novelty 4 | "0.712 — beats a tuned ML model's 0.593"            | "0.712 recall-first F1 on 22 unseen projects; a tuned model that reached CV F1 0.816 fell to 0.593" |
| README "Key Results"                           | "Detection accuracy: graph vs naive 1.000 vs 0.947" | "No accuracy loss vs naive prompting (1.000 vs 0.947, n = 36, not significant)"                     |
| IEEE report §8.4                               | "the system catches every delayed sprint"           | add: "guaranteed by the label definition at sprint-end evaluation"                                  |
| IEEE report §8.4                               | "the interval strengthens the headline claim"       | remove; state the interval without a comparison, or run a paired test                               |
| IEEE report §8.7                               | "~92% (37/40)"                                      | "estimated 54–95%; 22 of 24 diagnosed failures were rate limits"                                    |
| IEEE report §9, limitation 7                   | "No confidence intervals"                           | out of date: intervals were added in §8.4                                                           |
| IEEE report §9, limitation 8                   | "RBAC not enforced"                                 | out of date: role permissions are now enforced                                                      |
| IEEE report §8.1                               | test counts 29 / 43 / 27                            | 52 / 59 / 41                                                                                        |
| IEEE report §8.4                               | end-of-sprint results as the real-world headline    | lead with the mid-sprint result (§16); keep end-of-sprint as a sanity check                         |

## Experiments that would turn the weak claims into strong ones

1. **Mid-sprint prediction.** Done: see §16.
2. **Paired tests on the held-out set.** Done: see §17.
3. **Fairer threshold.** Compare the composite at its F1-optimal threshold from the tuning split.
4. **A larger, stricter ablation.** More, larger scenarios; extra reported risks count as errors.
5. **Token and anomaly sweep with a real tokenizer.** Vary the number of anomalies as well as the project size, and count tokens exactly.
6. **Complete live-LLM diagnostics.** Re-run with per-call error logging to replace the 54–95% range with one number.
7. **A human evaluation of explanation usefulness.** Grounding shows citations are real, not that they help.